

EXPERIMENT – 3

AIM: To design and implement an Employee–Department–Project database schema and perform SQL queries using Selection, Projection, Aggregate Functions, GROUP BY, HAVING, CASE expressions, and ORDER BY.

SOURCE CODE:

DATABASE CREATING

```
CREATE DATABASE EmployeeManagement;  
USE EmployeeManagement;
```

TABLE CREATING

```
CREATE TABLE Department (  
    dept_id INT PRIMARY KEY,  
    dept_name VARCHAR(50) NOT NULL,  
    location VARCHAR(50) );  
  
CREATE TABLE Employee (  
    emp_id INT PRIMARY KEY,  
    emp_name VARCHAR(50) NOT NULL,  
    gender CHAR(1), salary DECIMAL(10,2),  
    hire_date DATE,  
    dept_id INT,  
    FOREIGN KEY (dept_id)  
        REFERENCES Department(dept_id)  
);  
  
CREATE TABLE Project (  
    project_id INT PRIMARY KEY,  
    project_name VARCHAR(100) NOT NULL,  
    budget DECIMAL(12,2),  
    dept_id INT,  
    FOREIGN KEY (dept_id)  
        REFERENCES Department(dept_id)  
);  
  
CREATE TABLE Employee_Project (  
    emp_id INT,  
    project_id INT,  
    hours_worked INT,  
    PRIMARY KEY (emp_id, project_id),  
    FOREIGN KEY (emp_id)  
        REFERENCES Employee(emp_id),
```

```
FOREIGN KEY (project_id)
REFERENCES Project(project_id)
);
```

INSERTING DEPARTMENTS

```
INSERT INTO Department (dept_id, dept_name, location)
VALUES
(1, 'Development', 'Delhi'),
(2, 'Human Resources', 'Mumbai'),
(3, 'Accounting', 'Bangalore'),
(4, 'Sales', 'Pune'),
(5, 'Innovation', 'Hyderabad');
```

INSERTING 30 EMPLOYEES

```
INSERT INTO Employee
(emp_id, emp_name, gender, salary, hire_date, dept_id)
VALUES
(101, 'Aarush Khanna', 'M', 78000, '2021-01-15', 1),
(102, 'Ishita Roy', 'F', 69000, '2022-03-10', 1),
(103, 'Vivaan Sinha', 'M', 84000, '2020-07-20', 1),
(104, 'Myra Joshi', 'F', 74000, '2021-11-05', 1),
(105, 'Reyansh Kapoor', 'M', 97000, '2019-06-18', 1),
(106, 'Anika Bhatia', 'F', 66000, '2023-01-12', 1),
(107, 'Kabir Malhotra', 'M', 57000, '2022-05-15', 2),
(108, 'Aadhya Menon', 'F', 64000, '2021-09-22', 2),
(109, 'Vihaan Arora', 'M', 60000, '2020-04-11', 2),
(110, 'Kiara Saxena', 'F', 69000, '2023-02-19', 2),
(111, 'Advait Rao', 'M', 62000, '2022-08-01', 2),
(112, 'Saanvi Desai', 'F', 73000, '2019-12-10', 2),
(113, 'Dhruv Mehta', 'M', 90000, '2020-01-25', 3),
(114, 'Navya Pillai', 'F', 94000, '2019-08-14', 3),
(115, 'Arnav Kulkarni', 'M', 78000, '2021-06-30', 3),
(116, 'Tara Chawla', 'F', 87000, '2022-10-17', 3),
(117, 'Ishaan Nair', 'M', 75000, '2023-03-21', 3),
(118, 'Riddhi Shah', 'F', 92000, '2020-11-09', 3),
(119, 'Krish Bansal', 'M', 72000, '2021-02-28', 4),
(120, 'Avni Khurana', 'F', 68000, '2022-04-15', 4),
(121, 'Rudra Mishra', 'M', 76000, '2020-09-12', 4),
(122, 'Pari Anand', 'F', 83000, '2019-07-07', 4),
(123, 'Yuvaan Sethi', 'M', 71000, '2023-01-05', 4),
(124, 'Diya Oberoi', 'F', 79000, '2021-12-18', 4),
(125, 'Atharv Reddy', 'M', 99000, '2018-05-20', 5),
(126, 'Meher Kaur', 'F', 93000, '2020-03-16', 5),
(127, 'Shaurya Das', 'M', 89000, '2021-10-08', 5),
(128, 'Zoya Fernandes', 'F', 95000, '2019-11-25', 5),
```

```
(129, 'Ayaan Thakur', 'M', 81000, '2022-07-14', 5),  
(130, 'Vanya Kapoor', 'F', 98000, '2023-02-02', 5);
```

INSERTING 8 PROJECTS

```
INSERT INTO Project (project_id, project_name, budget, dept_id) VALUES  
(201, 'Smart Web Portal', 550000, 1),  
(202, 'Cloud Infrastructure', 800000, 1),  
(203, 'Staff Development Program', 300000, 2),  
(204, 'Budget Forecasting System', 650000, 3),  
(205, 'Online Sales Campaign', 480000, 4),  
(206, 'Customer Insights Study', 400000, 4),  
(207, 'Predictive Analytics Platform', 950000, 5),  
(208, 'Cybersecurity Framework', 1100000, 5);
```

ASSIGNING EMPLOYEES TO PROJECTS

```
INSERT INTO Employee_Project (emp_id, project_id, hours_worked)  
VALUES  
(101, 201, 120),  
(102, 201, 100),  
(103, 202, 150),  
(104, 202, 110),  
(105, 201, 130),  
(106, 202, 90),  
(107, 203, 100),  
(108, 203, 120),  
(109, 203, 80),  
(110, 203, 110),  
(111, 203, 90),  
(112, 203, 130),  
(113, 204, 140),  
(114, 204, 150),  
(115, 204, 120),  
(116, 204, 135),  
(117, 204, 100),  
(118, 204, 145),  
(119, 205, 110),  
(120, 205, 100),  
(121, 206, 120),  
(122, 205, 140),  
(123, 206, 90),  
(124, 206, 130),  
(125, 207, 160),  
(126, 207, 140),  
(127, 208, 150),
```

(128, 208, 160),
(129, 207, 120),
(130, 208, 135);

SQL QUERIES:

SELECTION

SELECT

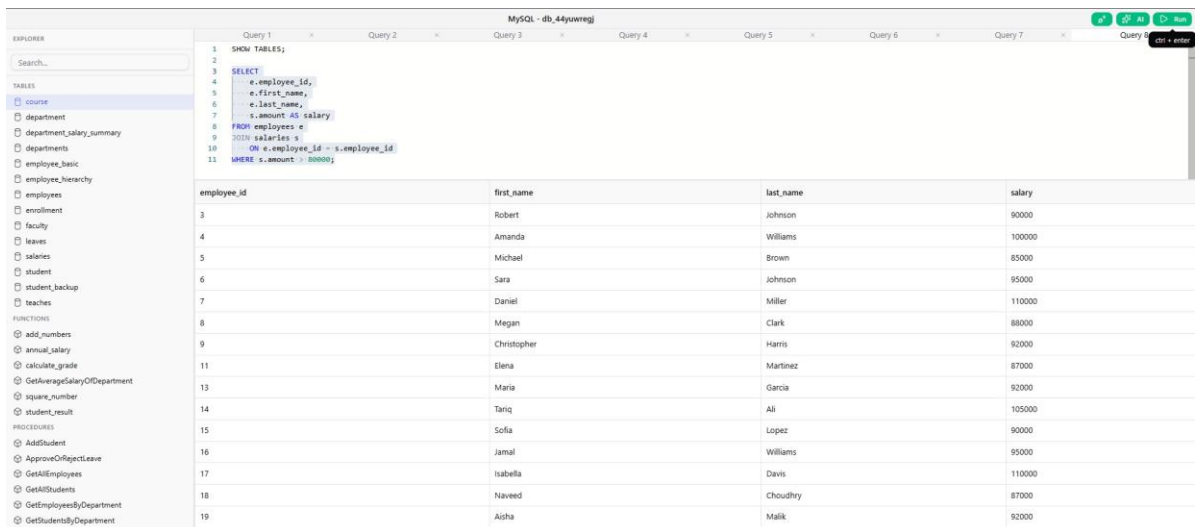
e.employee_id,
e.first_name,
e.last_name,
s.amount AS salary

FROM employees e

JOIN salaries s

ON e.employee_id = s.employee_id

WHERE s.amount > 80000;



The screenshot shows a MySQL database interface with a query editor and a results table. The query editor contains the following SQL code:

```
1 SHOW TABLES;
2
3 SELECT
4   e.employee_id,
5   e.first_name,
6   e.last_name,
7   s.amount AS salary
8 FROM employees e
9 JOIN salaries s
10 ON e.employee_id = s.employee_id
11 WHERE s.amount > 80000;
```

The results table displays the following data:

employee_id	first_name	last_name	salary
3	Robert	Johnson	90000
4	Amanda	Williams	100000
5	Michael	Brown	85000
6	Sara	Johnson	95000
7	Daniel	Miller	110000
8	Megan	Clark	88000
9	Christopher	Harris	92000
11	Elena	Martinez	87000
13	Maria	Garcia	92000
14	Tariq	Ali	105000
15	Sofia	Lopez	90000
16	Jamal	Williams	95000
17	Isabella	Davis	110000
18	Naveed	Choudhry	87000
19	Aisha	Malik	92000

PROJECTION

SELECT

CONCAT(e.first_name, ' ', e.last_name) AS emp_name,
s.amount AS salary

FROM employees e

JOIN salaries s

ON e.employee_id = s.employee_id;

MySQL - db_44yuregi																																									
Search... TABLES course department department_salary_summary departments employee_basic employee_hierarchy employees enrollment faculty leaves salaries student student_backup teaches FUNCTIONS add_numbers annual_salary calculate_grade GetAverageSalaryOfDepartment squre_number student_result PROCEDURES AddStudent ApproveOrRejectLeave GetAllEmployees GetAllStudents GetEmployeesByDepartment GetStudentsByDepartment	Query 1 Query 2 Query 3 Query 4 Query 5 Query 6 Query 7 Query 8 <pre> 12 SELECT 13 CONCAT(e.first_name, ' ', e.last_name) AS emp_name, 14 s.amount AS salary 15 FROM employees e 16 JOIN salaries s 17 ON e.employee_id = s.employee_id </pre> <table> <thead> <tr> <th>emp_name</th><th>salary</th></tr> </thead> <tbody> <tr><td>John Doe</td><td>80000</td></tr> <tr><td>Jane Smith</td><td>75000</td></tr> <tr><td>Robert Johnson</td><td>90000</td></tr> <tr><td>Amanda Williams</td><td>100000</td></tr> <tr><td>Michael Brown</td><td>85000</td></tr> <tr><td>Sara Johnson</td><td>95000</td></tr> <tr><td>Daniel Miller</td><td>110000</td></tr> <tr><td>Megan Clark</td><td>88000</td></tr> <tr><td>Christopher Harris</td><td>92000</td></tr> <tr><td>Olivia White</td><td>80000</td></tr> <tr><td>Elena Martinez</td><td>87000</td></tr> <tr><td>Ahmed Khan</td><td>78000</td></tr> <tr><td>Maria Garcia</td><td>92000</td></tr> <tr><td>Tariq Ali</td><td>105000</td></tr> <tr><td>Sofia Lopez</td><td>90000</td></tr> <tr><td>Jamal Williams</td><td>95000</td></tr> <tr><td>Isabella Davis</td><td>110000</td></tr> <tr><td>Naveed Choudhry</td><td>87000</td></tr> <tr><td>Aisha Malik</td><td>92000</td></tr> </tbody> </table>	emp_name	salary	John Doe	80000	Jane Smith	75000	Robert Johnson	90000	Amanda Williams	100000	Michael Brown	85000	Sara Johnson	95000	Daniel Miller	110000	Megan Clark	88000	Christopher Harris	92000	Olivia White	80000	Elena Martinez	87000	Ahmed Khan	78000	Maria Garcia	92000	Tariq Ali	105000	Sofia Lopez	90000	Jamal Williams	95000	Isabella Davis	110000	Naveed Choudhry	87000	Aisha Malik	92000
emp_name	salary																																								
John Doe	80000																																								
Jane Smith	75000																																								
Robert Johnson	90000																																								
Amanda Williams	100000																																								
Michael Brown	85000																																								
Sara Johnson	95000																																								
Daniel Miller	110000																																								
Megan Clark	88000																																								
Christopher Harris	92000																																								
Olivia White	80000																																								
Elena Martinez	87000																																								
Ahmed Khan	78000																																								
Maria Garcia	92000																																								
Tariq Ali	105000																																								
Sofia Lopez	90000																																								
Jamal Williams	95000																																								
Isabella Davis	110000																																								
Naveed Choudhry	87000																																								
Aisha Malik	92000																																								

AGGREGATE FUNCTIONS

SELECT

COUNT(DISTINCT employee_id) AS total_employees,

SUM(amount) AS total_salary,

AVG(amount) AS average_salary,

MAX(amount) AS highest_salary,

MIN(amount) AS lowest_salary

FROM salaries;

EXPLORER

Search...

TABLES

course

department

department_salary_summary

departments

employee_basic

employee_hierarchy

employees

MySQL - db_44yuregi

Query 1

Query 2

Query 3

Query 4

Query 5

Query 6

Query 7

Query 8

```
20 SELECT
21   COUNT(DISTINCT employee_id) AS total_employees,
22   SUM(amount) AS total_salary,
23   AVG(amount) AS average_salary,
24   MAX(amount) AS highest_salary,
25   MIN(amount) AS lowest_salary
26 FROM salaries;
```

total_employees	total_salary	average_salary	highest_salary	lowest_salary
20	1811000	90550.0000	110000	75000

GROUP BY

SELECT

dept_id,

COUNT(*) AS employee_count

FROM Employee

GROUP BY dept_id;

MySQL - db_44yuregi													
Search... TABLES course department department_salary_summary departments employee_basic employee_hierarchy employees enrollment faculty leaves salaries student student_backup	Query 1 Query 2 Query 3 Query 4 Query 5 Query 6 Query 7 Query 8 <pre> 28 FROM salaries; 29 30 SELECT 31 department_id, 32 COUNT(*) AS employee_count 33 FROM employees 34 GROUP BY department_id; </pre> <table> <thead> <tr> <th>department_id</th><th>employee_count</th></tr> </thead> <tbody> <tr><td>101</td><td>4</td></tr> <tr><td>102</td><td>4</td></tr> <tr><td>103</td><td>4</td></tr> <tr><td>104</td><td>4</td></tr> <tr><td>105</td><td>4</td></tr> </tbody> </table>	department_id	employee_count	101	4	102	4	103	4	104	4	105	4
department_id	employee_count												
101	4												
102	4												
103	4												
104	4												
105	4												

GROUP BY WITH JOIN

SELECT

d.department_name,
AVG(s.amount) AS average_salary

FROM departments d

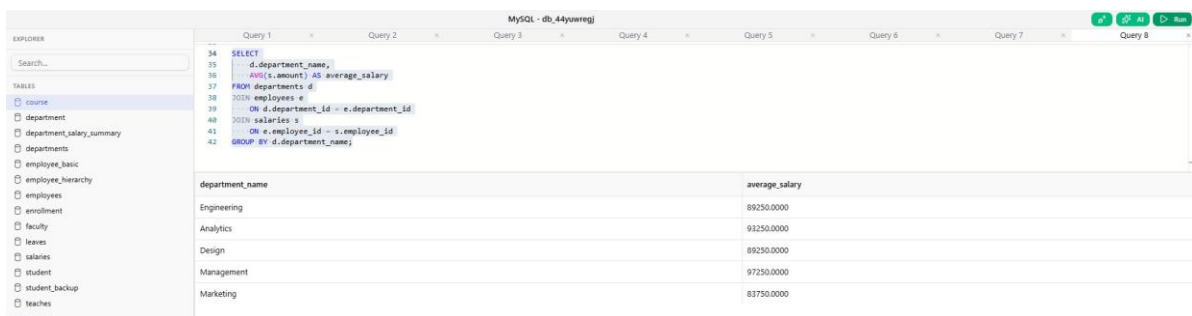
JOIN employees e

ON d.department_id = e.department_id

JOIN salaries s

ON e.employee_id = s.employee_id

GROUP BY d.department_name;



The screenshot shows the MySQL Workbench interface. The left sidebar lists various database tables. The main window displays a SQL query (Query 1) and its results. The query is a JOIN of departments, employees, and salaries tables, grouped by department name. The results table has two columns: department_name and average_salary.

department_name	average_salary
Engineering	89250.0000
Analytics	93250.0000
Design	89250.0000
Management	97250.0000
Marketing	83750.0000

HAVING

SELECT

d.department_name,
AVG(s.amount) AS average_salary

FROM departments d

JOIN employees e

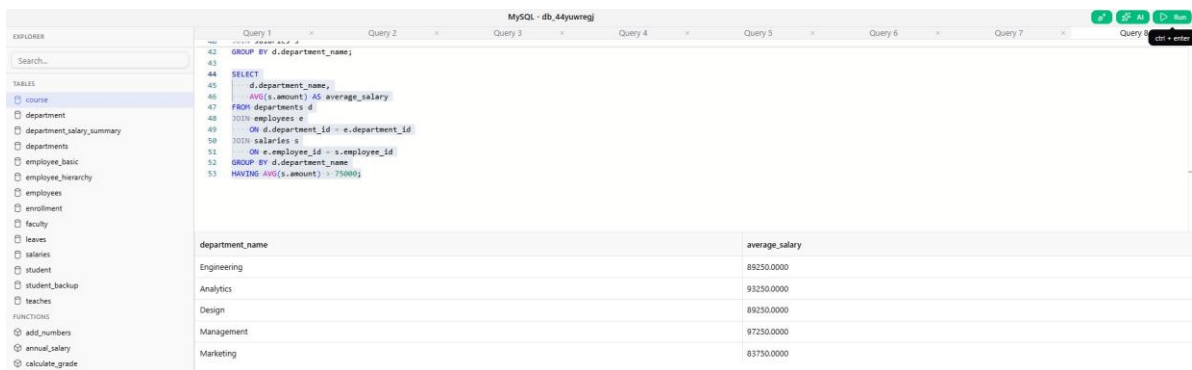
ON d.department_id = e.department_id

JOIN salaries s

ON e.employee_id = s.employee_id

GROUP BY d.department_name

HAVING AVG(s.amount) > 75000;



The screenshot shows the MySQL Workbench interface. The left sidebar lists various database tables. The main window displays a SQL query (Query 1) and its results. The query is a JOIN of departments, employees, and salaries tables, grouped by department name, with a HAVING clause filtering for average salary greater than 75000. The results table has two columns: department_name and average_salary.

department_name	average_salary
Engineering	89250.0000
Analytics	93250.0000
Design	89250.0000
Management	97250.0000
Marketing	83750.0000

CASE EXPRESSIONS

SELECT

CONCAT(e.first_name, ' ', e.last_name) AS emp_name,
s.amount AS salary,

CASE

WHEN s.amount >= 90000 THEN 'High Salary'

WHEN s.amount >= 70000 THEN 'Medium Salary'

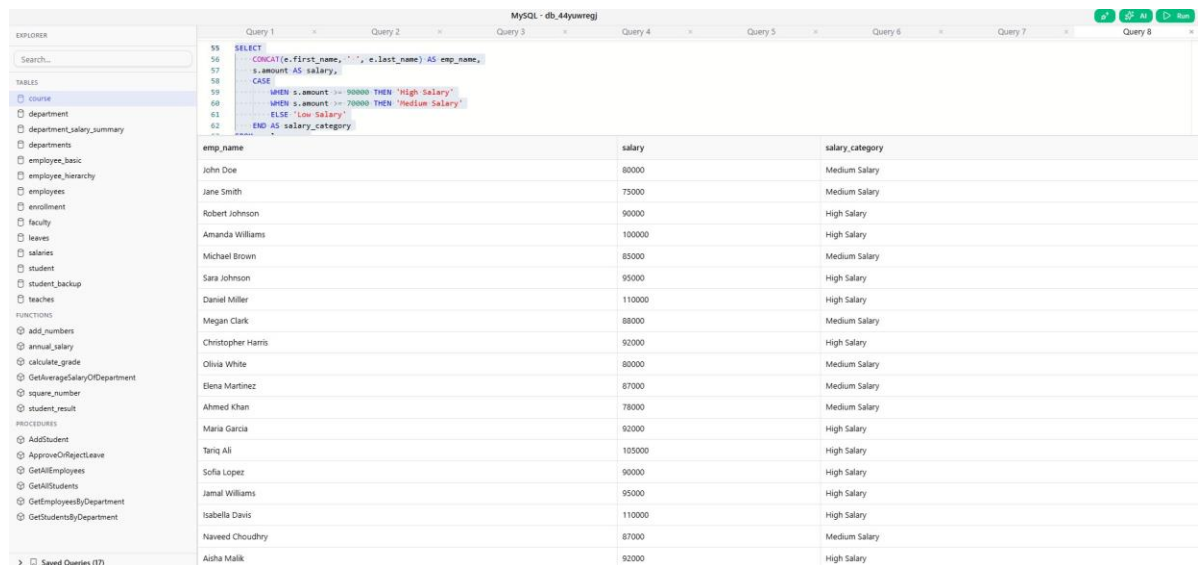
ELSE 'Low Salary'

END AS salary_category

FROM employees e

JOIN salaries s

ON e.employee_id = s.employee_id;



The screenshot shows a MySQL database interface with a query editor and a results table. The query editor contains the following SQL code:

```
55 SELECT
56   CONCAT(e.first_name, ' ', e.last_name) AS emp_name,
57   s.amount AS salary,
58   CASE
59     WHEN s.amount >= 90000 THEN 'High Salary'
60     WHEN s.amount >= 70000 THEN 'Medium Salary'
61     ELSE 'Low Salary'
62   END AS salary_category
```

The results table displays the following data:

emp_name	salary	salary_category
John Doe	80000	Medium Salary
Jane Smith	75000	Medium Salary
Robert Johnson	90000	High Salary
Amanda Williams	100000	High Salary
Michael Brown	85000	Medium Salary
Sara Johnson	95000	High Salary
Daniel Miller	110000	High Salary
Megan Clark	88000	Medium Salary
Christopher Harris	92000	High Salary
Olivia White	80000	Medium Salary
Elena Martinez	87000	Medium Salary
Ahmed Khan	78000	Medium Salary
Maria Garcia	92000	High Salary
Tariq Ali	105000	High Salary
Sofia Lopez	90000	High Salary
Jamal Williams	95000	High Salary
Isabella Davis	110000	High Salary
Naveed Choudhry	87000	Medium Salary
Aisha Malik	92000	High Salary

COMPLETE EMPLOYEE-DEPARTMENT REPORT

SELECT

p.project_id,

p.project_name,

d.department_name,

p.budget

FROM project p

JOIN departments d

ON p.dept_id = d.department_id

ORDER BY p.budget DESC;

EXPLORER

Search...

TABLES

course

department

department_salary_summary

departments

employee_basic

employee_hierarchy

employees

enrollment

faculty

leaves

salaries

student

student_backup

teaches

FUNCTIONS

add_numbers

annual_salary

calculate_grade

GetAverageSalaryOfDepartment

MySQL - db_44ywnregj

Query 1

Query 2

Query 3

Query 4

Query 5

Query 6

Query 7

Query 8

```
120  (info, 'Cybersecurity Framework', 1100000, 180);
121
122  SELECT
123    p.project_id,
124    p.project_name,
125    d.department_name,
126    p.budget
127  FROM project p
128    JOIN departments d
129    ON p.dept_id = d.department_id
130  ORDER BY p.budget DESC;
131
132
```

project_id	project_name	department_name	budget
208	Cybersecurity Framework	Engineering	1100000.00
207	Predictive Analytics Platform	Analytics	950000.00
202	Cloud infrastructure	Engineering	800000.00
204	Budget Forecasting System	Analytics	650000.00
201	Smart Web Portal	Engineering	550000.00
205	Online Sales Campaign	Marketing	480000.00
206	Customer Insights Study	Marketing	400000.00
203	Staff Development Program	Management	300000.00

PROJECT DETAILS

SELECT

 p.project_name,
 d.department_name,
 p.budget

FROM project p

JOIN departments d

 ON p.dept_id = d.department_id

ORDER BY p.budget DESC;

MySQL - db_44ywnregj

EXPLORER

Search...

TABLES

course

department

department_salary_summary

departments

employee_basic

employee_hierarchy

employees

enrollment

faculty

leaves

salaries

student

student_backup

teaches

FUNCTIONS

add_numbers

annual_salary

Query 1

Query 2

Query 3

Query 4

Query 5

Query 6

Query 7

Query 8

147

148

149

150

151

152

153

154

155

```
SELECT
  p.project_name,
  d.department_name,
  p.budget
FROM project p
JOIN departments d
  ON p.dept_id = d.department_id
ORDER BY p.budget DESC;
```

project_name

department_name

budget

Cybersecurity Framework

Engineering

1100000.00

Predictive Analytics Platform

Analytics

950000.00

Cloud Infrastructure

Engineering

800000.00

Budget Forecasting System

Analytics

650000.00

Smart Web Portal

Engineering

550000.00

Online Sales Campaign

Marketing

480000.00

Customer Insights Study

Marketing

400000.00

Staff Development Program

Management

300000.00

EMPLOYEES WORKING ON PROJECTS

SELECT

 CONCAT(e.first_name, ' ', e.last_name) AS emp_name,
 p.project_name,
 ep.hours_worked

FROM employee_project ep

JOIN employees e

 ON ep.employee_id = e.employee_id

JOIN project p

 ON ep.project_id = p.project_id

ORDER BY ep.hours_worked DESC;

MySQL - db_44juwregj

Query 1Query 2Query 3Query 4Query 5Query 6Query 7Query 8

Search...

TABLES

course

department

department_salary_summary

departments

employee_basic

employee_hierarchy

employees

enrollment

faculty

leaves

salaries

student

student_backup

teachers

FUNCTIONS

add_numbers

annual_salary

calculate_grade

GetAverageSalaryOfDepartment

square_number

student_result

PROCEDURES

AddStudent

ApproveOrRejectLeave

GetAllEmployees

GetAllStudents

GetEmployeesByDepartment

GetStudentsByDepartment

223 SELECT

224 FROM employee_project ep

225 JOIN employees e

226 ON ep.employee_id = e.employee_id

227 JOIN project p

228 ON ep.project_id = p.project_id

229 ORDER BY ep.hours_worked DESC

emp_name	project_name	hours_worked
Isabella Davis	Predictive Analytics Platform	160
Aisha Malik	Cybersecurity Framework	150
Robert Johnson	Cloud Infrastructure	150
Maria Garcia	Budget Forecasting System	150
Naveed Choudhry	Predictive Analytics Platform	140
Elena Martinez	Budget Forecasting System	140
Marcus Hill	Cybersecurity Framework	135
Ahmed Khan	Budget Forecasting System	130
Michael Brown	Smart Web Portal	130
Megan Clark	Staff Development Program	120
Jamal Williams	Customer Insights Study	120
John Doe	Smart Web Portal	120
Olivia White	Staff Development Program	110
Tariq Ali	Online Sales Campaign	110
Amanda Williams	Cloud Infrastructure	110
Daniel Miller	Staff Development Program	100
Sofia Lopez	Online Sales Campaign	100
Jane Smith	Smart Web Portal	100
Sara Johnson	Cloud Infrastructure	90

> Saved Queries (37)

EXPERIMENT – 4

AIM: To perform and analyze SQL queries using INNER JOIN, LEFT JOIN, SELF JOIN, 3way JOIN, correlated subqueries, EXISTS, and simulated INTERSECT and EXCEPT operations on the Employee–Department–Project database, and to compare their execution plans using EXPLAIN.

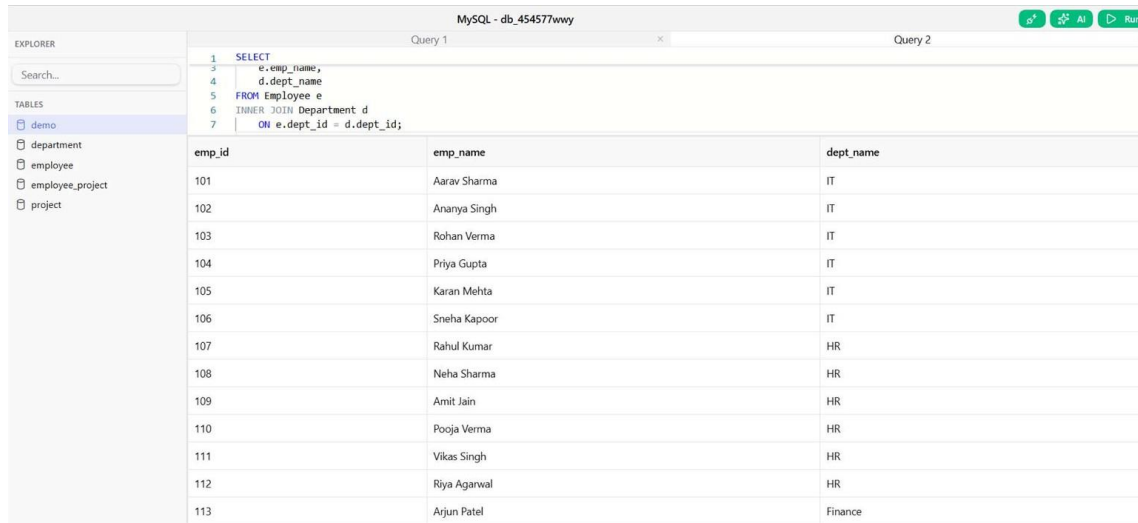
SOURCE CODE: USING EXPERIMENT – 3 DATABASE

SQL QUERIES:

INNER JOIN

```
SELECT
    e.emp_id,
    e.emp_name,
    d.dept_name
FROM Employee e
INNER JOIN Department d
    ON e.dept_id = d.dept_id;
```

OUTPUT:



MySQL - db_454577wwy

Query 1

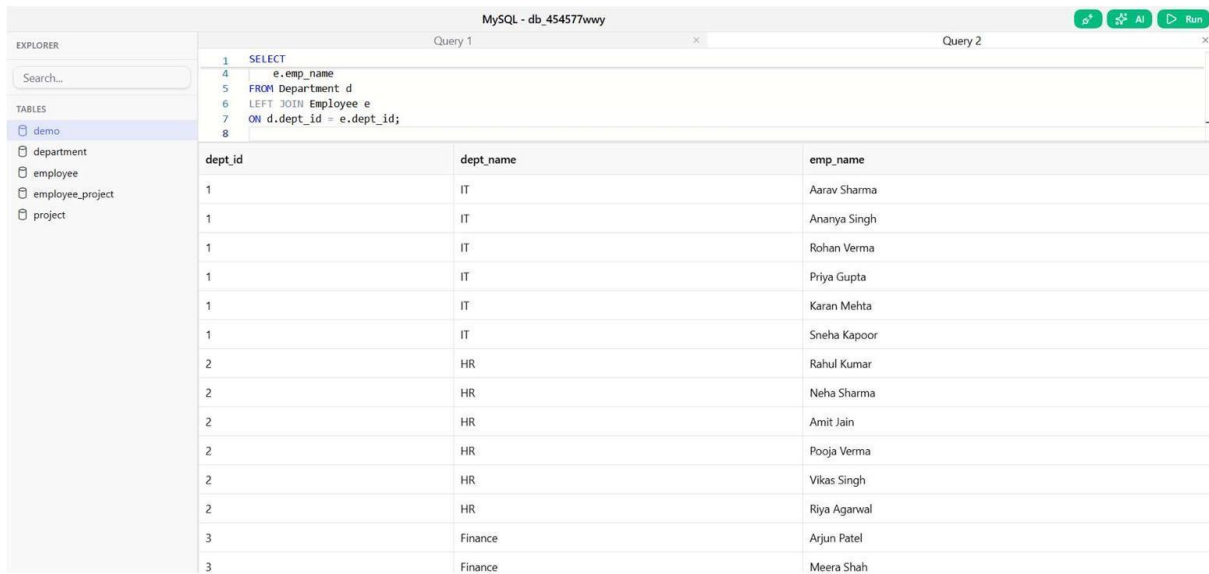
```
1 SELECT
2   e.emp_name,
3   d.dept_name
4 FROM Employee e
5 INNER JOIN Department d
6   ON e.dept_id = d.dept_id;
```

emp_id	emp_name	dept_name
101	Aarav Sharma	IT
102	Ananya Singh	IT
103	Rohan Verma	IT
104	Priya Gupta	IT
105	Karan Mehta	IT
106	Sineha Kapoor	IT
107	Rahul Kumar	HR
108	Neha Sharma	HR
109	Amit Jain	HR
110	Pooja Verma	HR
111	Vikas Singh	HR
112	Riya Agarwal	HR
113	Arjun Patel	Finance

LEFT JOIN

```
SELECT
    d.dept_id,
    d.dept_name,
    e.emp_name
FROM Department d
LEFT JOIN Employee e
    ON d.dept_id = e.dept_id;
```

OUTPUT:



The screenshot shows a MySQL IDE window titled 'MySQL - db_454577wwy'. The 'EXPLORER' panel on the left lists tables: department, employee, employee_project, and project. The 'Query 1' editor shows a SQL query:
`1 SELECT
4 e.emp_name
5 FROM Department d
6 LEFT JOIN Employee e
7 ON d.dept_id = e.dept_id;
8`
The 'Query 2' panel displays the result of this query as a table with three columns: dept_id, dept_name, and emp_name. The data is as follows:

dept_id	dept_name	emp_name
1	IT	Aarav Sharma
1	IT	Ananya Singh
1	IT	Rohan Verma
1	IT	Priya Gupta
1	IT	Karan Mehta
1	IT	Sneha Kapoor
2	HR	Rahul Kumar
2	HR	Neha Sharma
2	HR	Amit Jain
2	HR	Pooja Verma
2	HR	Vikas Singh
2	HR	Riya Agarwal
3	Finance	Arjun Patel
3	Finance	Meera Shah

3-WAY JOIN

SELECT

e.emp_name,
d.dept_name,
p.project_name

FROM Employee e

JOIN Department d

ON e.dept_id = d.dept_id

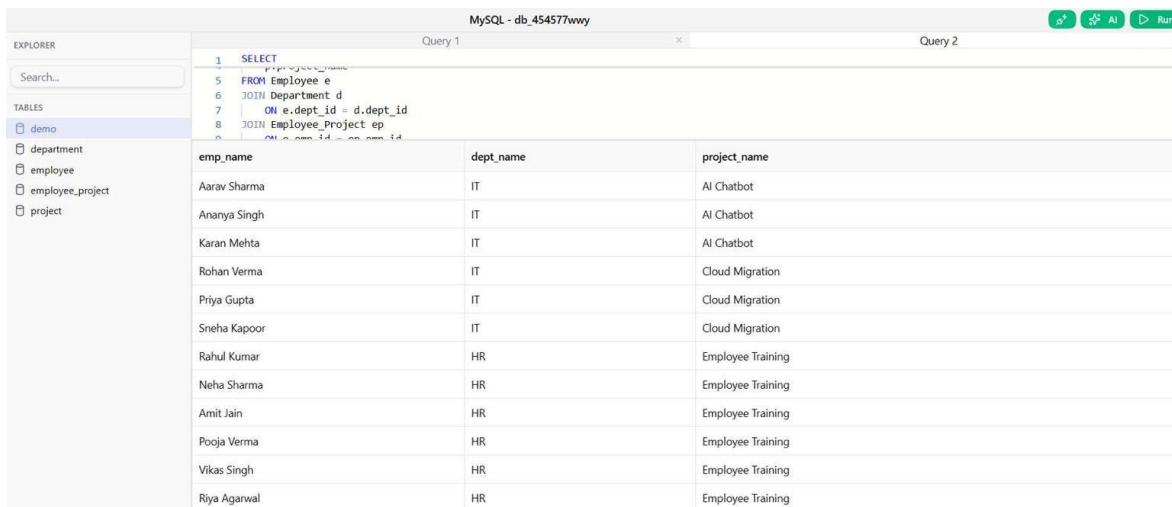
JOIN Employee_Project ep

ON e.emp_id = ep.emp_id

JOIN Project p

ON ep.project_id = p.project_id;

OUTPUT:



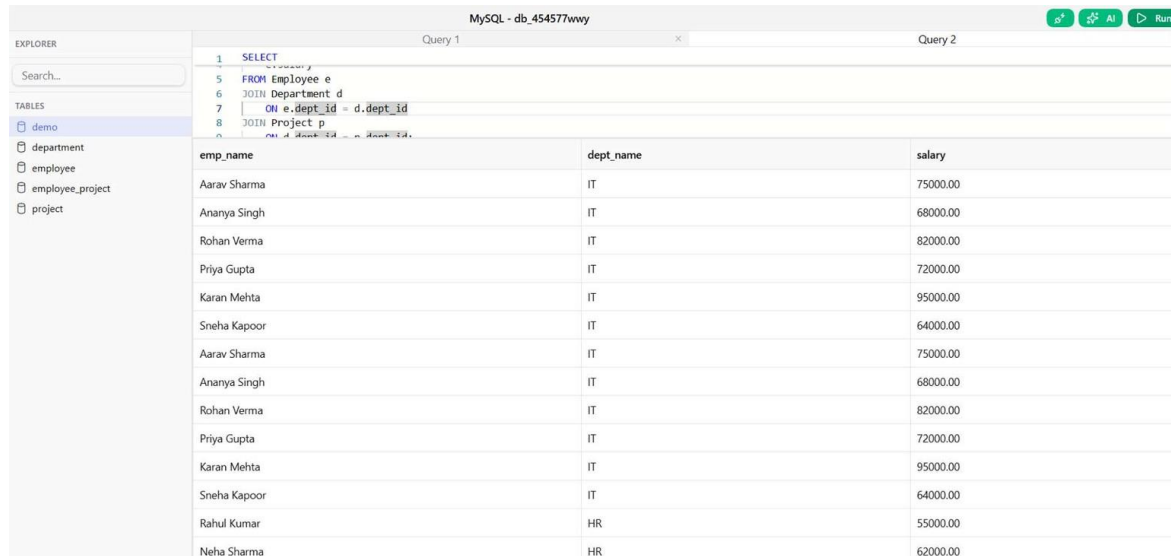
The screenshot shows a MySQL IDE window titled 'MySQL - db_454577wwy'. The 'EXPLORER' panel on the left lists tables: department, employee, employee_project, and project. The 'Query 1' editor shows a SQL query:
`1 SELECT
5 e.emp_name
6 FROM Employee e
7 JOIN Department d
8 ON e.dept_id = d.dept_id
9 JOIN Employee_Project ep
10 ON e.emp_id = ep.emp_id`
The 'Query 2' panel displays the result of this query as a table with three columns: emp_name, dept_name, and project_name. The data is as follows:

emp_name	dept_name	project_name
Aarav Sharma	IT	AI Chatbot
Ananya Singh	IT	AI Chatbot
Karan Mehta	IT	AI Chatbot
Rohan Verma	IT	Cloud Migration
Priya Gupta	IT	Cloud Migration
Sneha Kapoor	IT	Cloud Migration
Rahul Kumar	HR	Employee Training
Neha Sharma	HR	Employee Training
Amit Jain	HR	Employee Training
Pooja Verma	HR	Employee Training
Vikas Singh	HR	Employee Training
Riya Agarwal	HR	Employee Training

SELECT

```
    e.emp_name,  
    d.dept_name,  
    e.salary  
FROM Employee e  
JOIN Department d  
    ON e.dept_id = d.dept_id  
JOIN Project p  
    ON d.dept_id = p.dept_id;
```

OUTPUT:



The screenshot shows a MySQL IDE window titled 'MySQL - db_454577wwy'. On the left is an 'EXPLORER' pane with a search bar and a 'TABLES' list containing 'demo', 'department', 'employee', 'employee_project', and 'project'. The 'demo' table is selected. The main pane shows 'Query 1' with the following SQL code:

```
1 SELECT  
2     e.emp_name,  
3     d.dept_name,  
4     e.salary  
5 FROM Employee e  
6 JOIN Department d  
7     ON e.dept_id = d.dept_id  
8 JOIN Project p  
9     ON d.dept_id = p.dept_id;
```

Below the query, the results are displayed in a table with three columns: 'emp_name', 'dept_name', and 'salary'. The results are as follows:

emp_name	dept_name	salary
Aarav Sharma	IT	75000.00
Ananya Singh	IT	68000.00
Rohan Verma	IT	82000.00
Priya Gupta	IT	72000.00
Karan Mehta	IT	95000.00
Sneha Kapoor	IT	64000.00
Aarav Sharma	IT	75000.00
Ananya Singh	IT	68000.00
Rohan Verma	IT	82000.00
Priya Gupta	IT	72000.00
Karan Mehta	IT	95000.00
Sneha Kapoor	IT	64000.00
Rahul Kumar	HR	55000.00
Neha Sharma	HR	62000.00

CORRELATED SUBQUERY

SELECT

```
    e.emp_id, e.emp_name, e.salary, e.dept_id  
FROM Employee e  
WHERE e.salary >  
(  
    SELECT AVG(e2.salary)  
    FROM Employee e2  
    WHERE e2.dept_id = e.dept_id  
);
```

OUTPUT:

MySQL - db_454577wwy				
Query 1				
Query 2				
1	SELECT			
6	FROM Employee e			
7	WHERE e.salary >			
8	(			
9	SELECT AVG(e2.salary)			
10	FROM Employee e2			
11	);			
emp_id	emp_name	salary	dept_id	
103	Rohan Verma	82000.00	1	
105	Karan Mehta	95000.00	1	
110	Pooja Verma	67000.00	2	
112	Riya Agarwal	71000.00	2	
113	Arjun Patel	88000.00	3	
114	Meera Shah	92000.00	3	
116	Kavya Iyer	85000.00	3	
118	Nisha Reddy	90000.00	3	
121	Mohit Bansal	74000.00	4	
122	Isha Malhotra	81000.00	4	
124	Tanya Arora	77000.00	4	
125	Dev Malhotra	98000.00	5	
126	Aditi Mishra	91000.00	5	
128	Shreya Das	93000.00	5	

EXISTS

SELECT

```

        e.emp_id,
        e.emp_name
FROM Employee e
WHERE EXISTS
(
    SELECT 1
    FROM Employee_Project ep
    WHERE ep.emp_id = e.emp_id
);

```

OUTPUT:

MySQL - db_454577wwy				
Query 1				
Query 2				
4	FROM Employee e			
5	WHERE EXISTS			
6	(			
7	SELECT 1			
8	FROM Employee_Project ep			
9	WHERE ep.emp_id = e.emp_id			
10	);			
emp_id	emp_name			
101	Aarav Sharma			
102	Ananya Singh			
103	Rohan Verma			
104	Priya Gupta			
105	Karan Mehta			
106	Sneha Kapoor			
107	Rahul Kumar			
108	Neha Sharma			
109	Amit Jain			
110	Pooja Verma			
111	Vikas Singh			
112	Riya Agarwal			
113	Arjun Patel			

SIMULATED INTERSECT

SELECT

```

        e.emp_id,
        e.emp_name
FROM Employee e

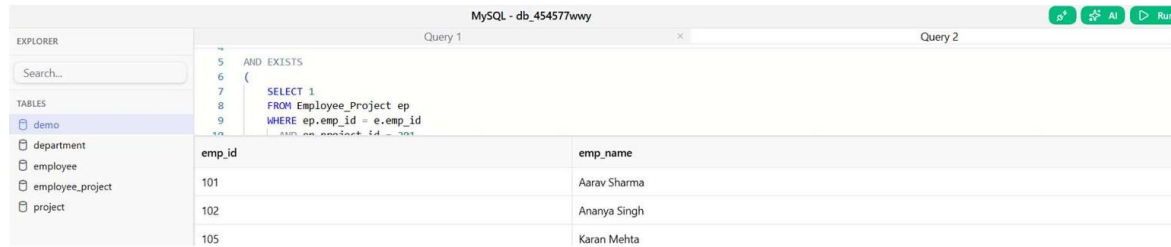
```

```

WHERE e.dept_id = 1
AND EXISTS
(
    SELECT 1
    FROM Employee_Project ep
    WHERE ep.emp_id = e.emp_id
    AND ep.project_id = 201
);

```

OUTPUT:



The screenshot shows a MySQL IDE window with a query editor and a results table. The query editor contains the following SQL code:

```

5 AND EXISTS
6 (
7     SELECT 1
8     FROM Employee_Project ep
9     WHERE ep.emp_id = e.emp_id
10    AND ep.project_id = 201

```

The results table displays the following data:

emp_id	emp_name
101	Aarav Sharma
102	Ananya Singh
105	Karan Mehta

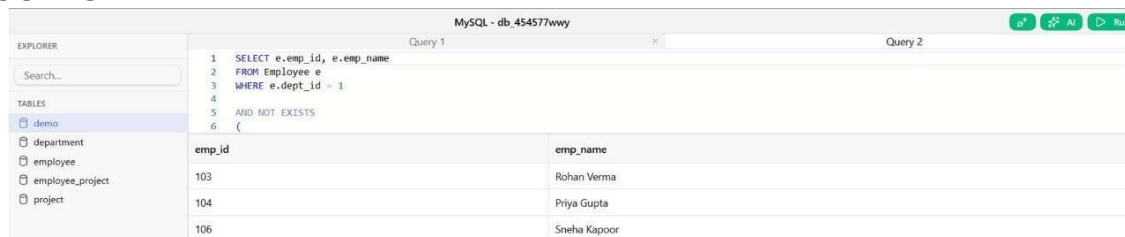
SIMULATED EXCEPT

```

SELECT e.emp_id, e.emp_name FROM Employee e
WHERE e.dept_id = 1
AND NOT EXISTS(
    SELECT 1
    FROM Employee_Project ep WHERE ep.emp_id = e.emp_id
    AND ep.project_id = 201
);

```

OUTPUT:



The screenshot shows a MySQL IDE window with a query editor and a results table. The query editor contains the following SQL code:

```

1 SELECT e.emp_id, e.emp_name
2 FROM Employee e
3 WHERE e.dept_id = 1
4
5 AND NOT EXISTS
6 (

```

The results table displays the following data:

emp_id	emp_name
103	Rohan Verma
104	Priya Gupta
106	Sneha Kapoor

COMPARING EXECUTION PLANS USING EXPLAIN EXPLAIN – INNER JOIN

EXPLAIN

```

SELECT
e.emp_id,
e.emp_name,
d.dept_name
FROM Employee e
INNER JOIN Department d
ON e.dept_id = d.dept_id;

```

OUTPUT:

MySQL - db_454577www												
Query 1												
1 EXPLAIN 2 SELECT 3 e.emp_id, 4 e.emp_name, 5 d.dept_name 6 FROM Employee e												
id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra	
1	SIMPLE	d	null	ALL	PRIMARY	null	null	null	5	100	null	
1	SIMPLE	e	null	ref	dept_id	dept_id	5	db_454577www.d.dept_id	6	100	null	

EXPLAIN – LEFT JOIN

EXPLAIN

SELECT

d.dept_name, e.emp_name

FROM Department d LEFT JOIN Employee e

ON d.dept_id = e.dept_id;

OUTPUT:

MySQL - db_454577www												
Query 1												
1 EXPLAIN 2 SELECT 3 d.dept_name, 4 e.emp_name 5 FROM Department d 6 LEFT JOIN Employee e												
id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra	
1	SIMPLE	d	null	ALL	null	null	null	null	5	100	null	
1	SIMPLE	e	null	ref	dept_id	dept_id	5	db_454577www.d.dept_id	6	100	null	

EXPLAIN – CORRELATED SUBQUERY

EXPLAIN SELECT

```
e.emp_name, e.salary
FROM Employee e
WHERE e.salary >
(
    SELECT AVG(e2.salary)
    FROM Employee e2
    WHERE e2.dept_id = e.dept_id
)
```

OUTPUT:

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	PRIMARY	e	null	ALL	null	null	null	null	30	100	Using where
2	DEPENDENT SUBQUERY	e2	null	ref	dept_id	dept_id	5	db_454577www.e.dept_id	6	100	null

EXPLAIN – EXISTS

EXPLAIN SELECT

```
e.emp_id, e.emp_name
FROM Employee e
WHERE EXISTS
(
    SELECT 1
    FROM Employee_Project ep
    WHERE ep.emp_id = e.emp_id
);
```

OUTPUT:

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	e	null	ALL	PRIMARY	null	null	null	30	100	null
1	SIMPLE	<subquery2>	null	eq_ref	<auto_distinct_key>	<auto_distinct_key>	4	db_454577www.e.emp_id	1	100	null
2	MATERIALIZED	ep	null	index	PRIMARY	project_id	4	null	30	100	Using index

EXPERIMENT – 5

AIM: To create and manipulate SQL views for department salary summary and employee hierarchy, test the updatability of views, and implement a recursive CTE to display employee reporting chains.

SOURCE CODE:

USING EXPERIMENT – 3 DATABASE

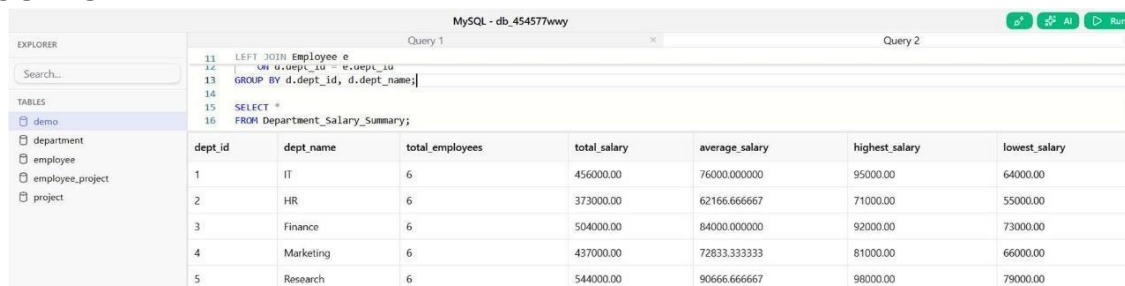
SQL QUERIES:

DEPARTMENT SALARY SUMMARY VIEW

CREATE VIEW

```
Department_Salary_Summary AS SELECT
    d.dept_id,
    d.dept_name,
    COUNT(e.emp_id) AS total_employees,
    SUM(e.salary) AS total_salary,
    AVG(e.salary) AS average_salary,
    MAX(e.salary) AS highest_salary,
    MIN(e.salary) AS lowest_salary
FROM Department d LEFT JOIN Employee e
    ON d.dept_id = e.dept_id
GROUP BY d.dept_id, d.dept_name;
SELECT *
FROM Department_Salary_Summary;
```

OUTPUT:

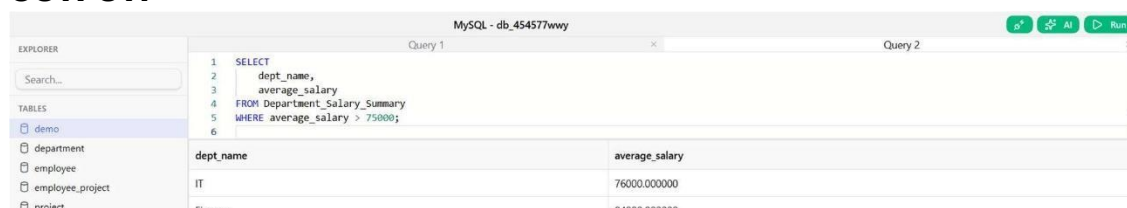


dept_id	dept_name	total_employees	total_salary	average_salary	highest_salary	lowest_salary
1	IT	6	456000.00	76000.000000	95000.00	64000.00
2	HR	6	373000.00	62166.666667	71000.00	55000.00
3	Finance	6	504000.00	84000.000000	92000.00	73000.00
4	Marketing	6	437000.00	72833.333333	81000.00	66000.00
5	Research	6	544000.00	90666.666667	98000.00	79000.00

QUERY THE VIEW

```
--Find departments with an average salary greater than ₹75,000 SELECT
dept_name, average_salary
FROM Department_Salary_Summary WHERE average_salary > 75000
```

OUTPUT:



dept_name	average_salary
IT	76000.000000

EMPLOYEE HIERARCHY

```
ALTER TABLE Employee
ADD manager_id INT NULL;
ALTER TABLE Employee
ADD CONSTRAINT fk_employee_manager FOREIGN KEY (manager_id)
REFERENCES Employee(emp_id);
```

INSERT EMPLOYEE HIERARCHY

-- Update Manager IDs

```
UPDATE Employee SET manager_id = NULL WHERE emp_id IN (105, 112, 114, 122,
125);
```

-- IT

```
UPDATE Employee
SET manager_id = 105
WHERE emp_id IN (103, 104);
UPDATE Employee
SET manager_id = 103
WHERE emp_id IN (101, 102);
UPDATE Employee
SET manager_id = 104
WHERE emp_id = 106;
```

-- HR

```
UPDATE Employee
SET manager_id = 105
WHERE emp_id IN (103, 104)
UPDATE Employee
SET manager_id = 103
WHERE emp_id IN (101, 102);
UPDATE Employee
SET manager_id = 104 WHERE emp_id = 106;
```

-- Finance UPDATE Employee

```
SET manager_id = 112
WHERE emp_id IN (107, 108);
UPDATE Employee
SET manager_id = 107 WHERE emp_id = 109;
UPDATE Employee
SET manager_id = 108
WHERE emp_id IN (110, 111)
```

-- Marketing

```
UPDATE Employee
SET manager_id = 112
```

```
UPDATE Employee
SET manager_id = 107 WHERE emp_id = 109;
UPDATE Employee
SET manager_id = 108
WHERE emp_id IN (110, 111);
```

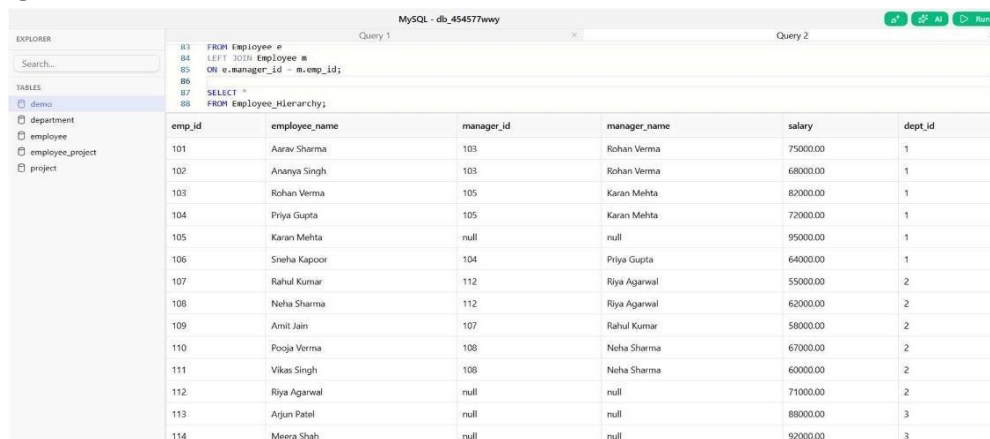
-- Research UPDATE Employee

```
SET manager_id = 125
WHERE emp_id IN (126, 127, 128);
UPDATE Employee
SET manager_id = 126
WHERE emp_id = 129;
UPDATE Employee
SET manager_id = 128 WHERE emp_id = 13
```

EMPLOYEE HIERARCHY VIEW

```
CREATE VIEW Employee_Hierarchy AS SELECT
e.emp_id,
e.emp_name AS employee_name, e.manager_id,
m.emp_name AS manager_name, e.salary,
e.dept_id FROM Employee e
LEFT JOIN Employee m
ON e.manager_id = m.emp_id;
SELECT *
FROM Employee_Hierarchy;
```

OUTPUT:



emp_id	employee_name	manager_id	manager_name	salary	dept_id
101	Aarav Sharma	103	Rohan Verma	75000.00	1
102	Ananya Singh	103	Rohan Verma	68000.00	1
103	Rohan Verma	105	Karan Mehta	82000.00	1
104	Priya Gupta	105	Karan Mehta	72000.00	1
105	Karan Mehta	null	null	95000.00	1
106	Sneha Kapoor	104	Priya Gupta	64000.00	1
107	Rahul Kumar	112	Riya Agarwal	55000.00	2
108	Neha Sharma	112	Riya Agarwal	62000.00	2
109	Amit Jain	107	Rahul Kumar	58000.00	2
110	Pooja Verma	108	Neha Sharma	67000.00	2
111	Vikas Singh	108	Neha Sharma	60000.00	2
112	Riya Agarwal	null	null	71000.00	2
113	Ajgun Patel	null	null	88000.00	3
114	Meera Shah	null	null	92000.00	3

OUTPUT:

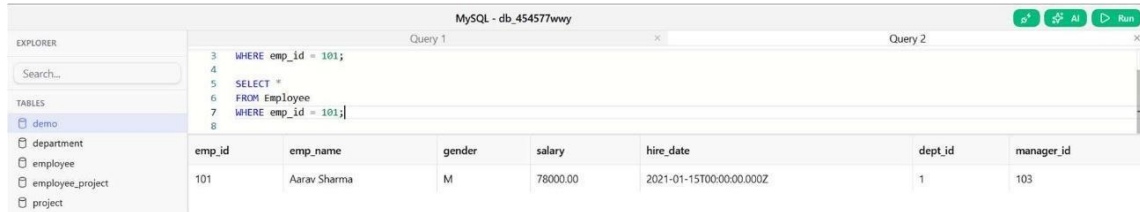


emp_id	emp_name	salary	dept_id
101	Aarav Sharma	75000.00	1

UPDATE THROUGH THE VIEW

```
UPDATE Employee_Basic SET salary = 78000 WHERE emp_id = 101;  
SELECT *  
FROM Employee WHERE emp_id = 1
```

OUTPUT:



The screenshot shows the MySQL Workbench interface. On the left is the Explorer pane with a search bar and a list of tables: demo, department, employee, employee_project, and project. The 'demo' table is selected. The main area displays 'Query 1' with the following SQL code:

```
3 WHERE emp_id = 101;  
4  
5 SELECT *  
6 FROM Employee  
7 WHERE emp_id = 101;  
8
```

Below the query, the results of the query are shown in a table with 8 columns: emp_id, emp_name, gender, salary, hire_date, dept_id, and manager_id. The results table contains one row of data for emp_id 101.

emp_id	emp_name	gender	salary	hire_date	dept_id	manager_id
101	Aarav Sharma	M	78000.00	2021-01-15T00:00:00.000Z	1	103

TEST NON-UPDATABILITY OF DEPARTMENT SALARY VIEW

```
UPDATE Department_Salary_Summary  
SET average_salary = 80000  
WHERE dept_id = 1;
```

OUTPUT:



The screenshot shows the MySQL Workbench interface. The main area displays 'Query 1' with the following SQL code:

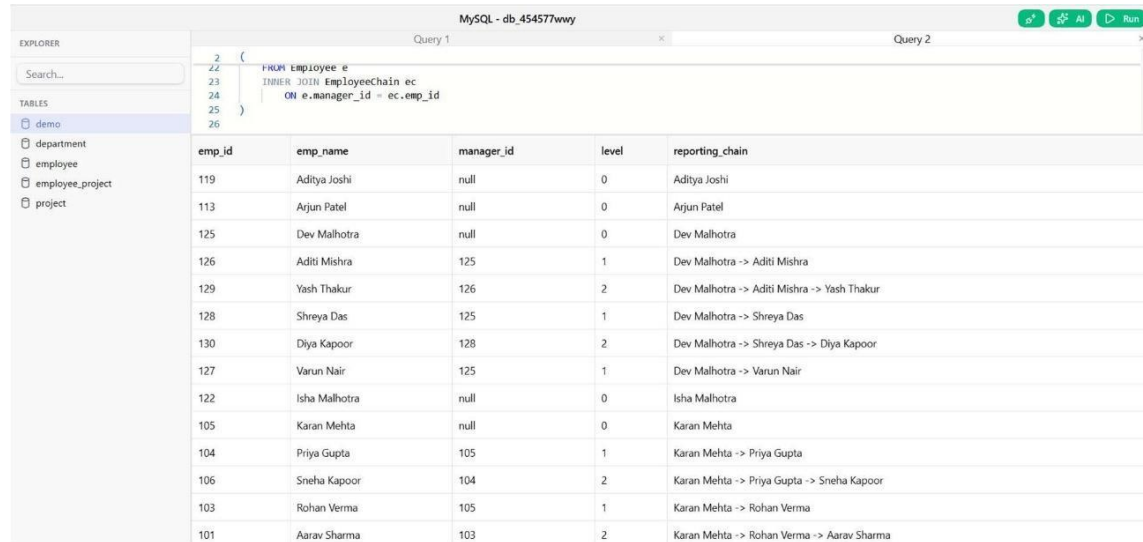
```
1 UPDATE Department_Salary_Summary  
2 SET average_salary = 80000  
3 WHERE dept_id = 1;  
4
```

Below the query, an error message is displayed: "Error: The target table Department_Salary_Summary of the UPDATE is not updatable."

RECURSIVE CTE

```
WITH RECURSIVE EmployeeChain AS (  
-- Anchor member  
SELECT  
emp_id, emp_name, manager_id,  
0 AS level,  
CAST(emp_name AS CHAR(500)) AS reporting_chain FROM Employee  
WHERE manager_id IS NULL UNION ALL  
-- Recursive member  
SELECT  
e.emp_id, e.emp_name, e.manager_id, ec.level + 1,  
CONCAT(ec.reporting_chain, ' -> ', e.emp_name) FROM Employee e  
INNER JOIN EmployeeChain ec  
ON e.manager_id = ec.emp_id  
)  
SELECT  
emp_id, emp_name, manager_id, level, reporting_chain  
FROM EmployeeChain  
ORDER BY reporting_chain;
```

OUTPUT:



The screenshot shows a MySQL IDE window with a query editor and a results pane. The query editor contains a recursive CTE query. The results pane shows the output of the query, which is a table with 5 columns: emp_id, emp_name, manager_id, level, and reporting_chain. The table contains 15 rows of data, showing the reporting chain for each employee starting from the root employee (emp_id 101).

emp_id	emp_name	manager_id	level	reporting_chain
119	Aditya Joshi	null	0	Aditya Joshi
113	Arjun Patel	null	0	Arjun Patel
125	Dev Malhotra	null	0	Dev Malhotra
126	Aditi Mishra	125	1	Dev Malhotra -> Aditi Mishra
129	Yash Thakur	126	2	Dev Malhotra -> Aditi Mishra -> Yash Thakur
128	Shreya Das	125	1	Dev Malhotra -> Shreya Das
130	Diya Kapoor	128	2	Dev Malhotra -> Shreya Das -> Diya Kapoor
127	Varun Nair	125	1	Dev Malhotra -> Varun Nair
122	Isha Malhotra	null	0	Isha Malhotra
105	Karan Mehta	null	0	Karan Mehta
104	Priya Gupta	105	1	Karan Mehta -> Priya Gupta
106	Sneha Kapoor	104	2	Karan Mehta -> Priya Gupta -> Sneha Kapoor
103	Rohan Verma	105	1	Karan Mehta -> Rohan Verma
101	Aarav Sharma	103	2	Karan Mehta -> Rohan Verma -> Aarav Sharma

RECURSIVE CTE FOR ONE EMPLOYEE

WITH RECURSIVE ReportingChain AS (

SELECT

emp_id, emp_name, manager_id,

0 AS level FROM Employee WHERE emp_id = 101

UNION ALL

SELECT

e.emp_id, e.emp_name, e.manager_id, rc.level + 1

FROM Employee e

JOIN ReportingChain rc

ON e.emp_id = rc.manager_id

)

SELECT

emp_id, emp_name,

manager_id, level

FROM ReportingChain;

OUTPUT:



The screenshot shows a MySQL IDE window with a query editor and a results pane. The query editor contains a recursive CTE query. The results pane shows the output of the query, which is a table with 4 columns: emp_id, emp_name, manager_id, and level. The table contains 3 rows of data, showing the reporting chain for employee 101.

emp_id	emp_name	manager_id	level
101	Aarav Sharma	103	0
103	Rohan Verma	105	1
105	Karan Mehta	null	2

EXPERIMENT – 6

AIM: To create a stored procedure `transfer_employee(emp_id, new_dept_id)` with validation and error handling, and to implement database triggers for salary validation and audit logging. The procedure and triggers are tested using normal and edge-case inputs.

SOURCE CODE:

USING EXPERIMENT – 3 DATABASE

SQL QUERIES:

STORED PROCEDURE: `transfer_employee`

The procedure will:

1. Check whether the employee exists.
2. Check whether the new department exists.
3. Check whether the employee is already in that department.
4. Transfer the employee.
5. Handle errors using `SQLEXCEPTION`.
6. Roll back the transaction if an error occurs.

```
DELIMITER //
```

```
CREATE PROCEDURE transfer_employee(  
    IN p_emp_id INT,  
    IN p_new_dept_id INT  
)  
BEGIN  
    DECLARE v_employee_count INT DEFAULT 0;  
    DECLARE v_department_count INT DEFAULT 0;  
    DECLARE v_current_dept INT DEFAULT 0;  
  
    -- Error handler  
    DECLARE EXIT HANDLER FOR SQLEXCEPTION  
    BEGIN  
        ROLLBACK;  
        SELECT 'Error occurred. Transaction rolled back.' AS  
message;  
        END;  
  
    START TRANSACTION;
```

```

-- Check whether employee exists
SELECT COUNT(*)
  INTO v_employee_count
  FROM Employee
  WHERE emp_id = p_emp_id;

IF v_employee_count = 0 THEN
  ROLLBACK;
  SELECT 'Error: Employee does not exist.' AS message;
ELSE
  -- Check whether department exists
  SELECT COUNT(*)
    INTO v_department_count
    FROM Department
    WHERE dept_id = p_new_dept_id;

  IF v_department_count = 0 THEN
    ROLLBACK;
    SELECT 'Error: Department does not exist.' AS
message;

  ELSE
    -- Get current department
    SELECT dept_id
      INTO v_current_dept
      FROM Employee
      WHERE emp_id = p_emp_id;
    -- Check if already in requested department
    IF v_current_dept = p_new_dept_id THEN
      ROLLBACK;
      SELECT 'Error: Employee is already in this
department.'
        AS message;

    ELSE
      -- Transfer employee
      UPDATE Employee
      SET dept_id = p_new_dept_id
      WHERE emp_id = p_emp_id;

      COMMIT;

      SELECT
        'Employee transferred successfully.'
        AS message;

      END IF;
    END IF;
  END IF;
END //
DELIMITER

```

TESTING THE STORED PROCEDURE

```
SELECT
    e.emp_id,
    e.emp_name,
    d.dept_name
FROM Employee e
JOIN Department d
    ON e.dept_id = d.dept_id
WHERE e.emp_id = 101;
```

OUTPUT:



The screenshot shows the MySQL IDE interface. The 'EXPLORER' pane on the left lists tables: demo, department, department_salary_summary, employee, employee_basic, and employee_hierarchy. The 'Query 1' pane displays the SQL query. The 'Query 2' pane shows the result set with three columns: emp_id, emp_name, and dept_name. The result row shows emp_id 101, emp_name 'Aarav Sharma', and dept_name 'IT'.

emp_id	emp_name	dept_name
101	Aarav Sharma	IT

```
CALL transfer_employee(101, 3);
```

OUTPUT:



The screenshot shows the MySQL IDE interface. The 'Query 1' pane displays the SQL query: CALL transfer_employee(101, 3);. The 'Query 2' pane shows the result set with one column: message. The result row shows the message 'Employee transferred successfully.'.

message
Employee transferred successfully.

```
SELECT
    e.emp_id,
    e.emp_name,
    d.dept_name
FROM Employee e
JOIN Department d
    ON e.dept_id = d.dept_id
WHERE e.emp_id = 101;
```

OUTPUT:



The screenshot shows the MySQL IDE interface. The 'EXPLORER' pane on the left lists tables: demo, department, department_salary_summary, employee, employee_basic, and employee_hierarchy. The 'Query 1' pane displays the SQL query. The 'Query 2' pane shows the result set with three columns: emp_id, emp_name, and dept_name. The result row shows emp_id 101, emp_name 'Aarav Sharma', and dept_name 'Finance'.

emp_id	emp_name	dept_name
101	Aarav Sharma	Finance

EDGE CASE – EMPLOYEE DOES NOT EXIST

```
CALL transfer_employee(999, 3);
```

OUTPUT:



The screenshot shows the MySQL IDE interface. The 'Query 1' pane displays the SQL query: CALL transfer_employee(999, 3);. The 'Query 2' pane shows the result set with one column: message. The result row shows the message 'Error: Employee does not exist.'.

message
Error: Employee does not exist.

EDGE CASE – DEPARTMENT DOES NOT EXIST

```
CALL transfer_employee(101, 99);
```

OUTPUT:



EDGE CASE – SAME DEPARTMENT

```
CALL transfer_employee(101, 3);
```

OUTPUT:



SALARY VALIDATION TRIGGER

Let, salary must be ≥ 15000

```
DELIMITER //  
CREATE TRIGGER validate_employee_salary  
BEFORE INSERT ON Employee  
FOR EACH ROW  
BEGIN  
    IF NEW.salary < 15000 THEN  
        SIGNAL SQLSTATE '45000'  
        SET MESSAGE_TEXT = 'Salary must be at least 15000';  
    END IF;  
END // DELIMITER ;
```

TEST SALARY VALIDATION

```
INSERT INTO Employee  
(emp_id, emp_name, gender, salary, hire_date, dept_id)  
VALUES  
(131, 'Test Employee', 'M', 10000, '2026-09-25', 1);
```

OUTPUT:



TEST VALID SALARY

```
INSERT INTO Employee
(emp_id, emp_name, gender, salary, hire_date, dept_id)
VALUES
(131, 'Test Employee', 'M', 30000, '2026-09-25', 1);
```

```
SELECT
    emp_id,
    emp_name,
    salary
FROM Employee
WHERE emp_id = 131;
```

OUTPUT:



The screenshot shows the MySQL IDE interface. On the left is the 'EXPLORER' pane with a search bar and a list of tables: 'demo', 'department', 'department_salary_summary', and 'employee'. The 'demo' table is selected. The main area displays 'Query 1' with the following SQL code:

```
6 SELECT
7     emp_id,
8     emp_name,
9     salary
10  FROM Employee
11  WHERE emp_id = 131;
```

Below the SQL code is a table with the query results:

emp_id	emp_name	salary
131	Test Employee	30000.00

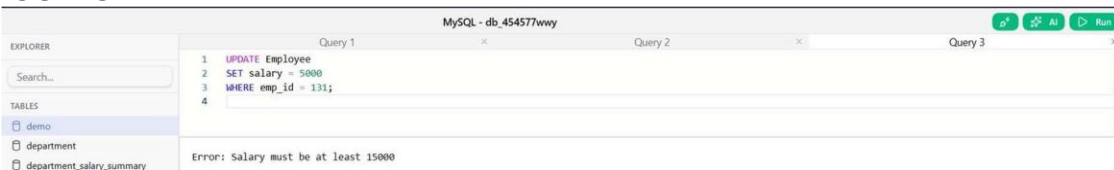
SALARY VALIDATION FOR UPDATE

```
DELIMITER //
CREATE TRIGGER validate_employee_salary_update
BEFORE UPDATE ON Employee
FOR EACH ROW
BEGIN
    IF NEW.salary < 15000 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Salary must be at least 15000';
    END IF;
END//
DELIMITER ;
```

TEST INVALID SALARY UPDATE

```
UPDATE Employee
SET salary = 5000
WHERE emp_id = 131;
```

OUTPUT:



The screenshot shows the MySQL IDE interface. The 'EXPLORER' pane is the same as in the previous screenshot. The main area displays 'Query 1' with the following SQL code:

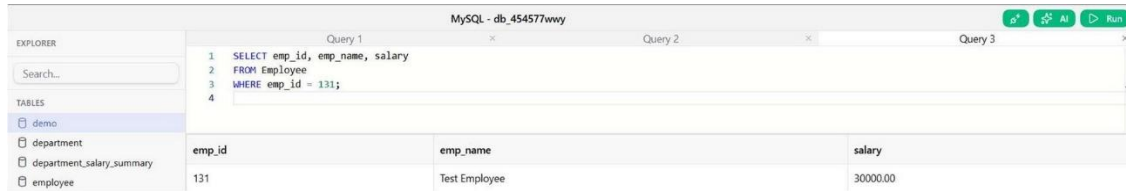
```
1 UPDATE Employee
2 SET salary = 5000
3 WHERE emp_id = 131;
4
```

Below the SQL code, an error message is displayed:

Error: Salary must be at least 15000

```
SELECT emp_id, emp_name, salary
FROM Employee
WHERE emp_id = 131;
```

OUTPUT:



The screenshot shows a MySQL IDE window titled 'MySQL - db_454577www'. On the left is an 'EXPLORER' pane with a search bar and a 'TABLES' list containing 'demo', 'department', 'department_salary_summary', and 'employee'. The main area displays 'Query 1' with the following SQL code:

```
1 SELECT emp_id, emp_name, salary
2 FROM Employee
3 WHERE emp_id = 131;
4
```

Below the query, the results are shown in a table with three columns: 'emp_id', 'emp_name', and 'salary'. The table contains one row of data.

emp_id	emp_name	salary
131	Test Employee	30000.00

CREATE AUDIT TABLE

```
CREATE TABLE Salary_Audit (
    audit_id INT AUTO_INCREMENT PRIMARY KEY,
    emp_id INT,
    old_salary DECIMAL(10,2),
    new_salary DECIMAL(10,2),
    changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    action VARCHAR(30)
);
```

AUDIT LOGGING TRIGGER

```
DELIMITER //
CREATE TRIGGER salary_audit_trigger
AFTER UPDATE ON Employee
FOR EACH ROW
BEGIN
    IF OLD.salary <> NEW.salary THEN
        INSERT INTO Salary_Audit
        (
            emp_id,
            old_salary,
            new_salary,
            action
        )
        VALUES
        (
            NEW.emp_id,
            OLD.salary,
            NEW.salary,
            'SALARY UPDATED'
        );
    END IF;
END //
DELIMITER ;
```

```

IF OLD.salary <> NEW.salary THEN
    INSERT INTO Salary_Audit
    (
        emp_id,
        old_salary,
        new_salary,
        action
    )
VALUES
    (
        NEW.emp_id,
        OLD.salary,
        NEW.salary,
        'SALARY UPDATED'
    );
END IF
END //
DELIMITER ;

```

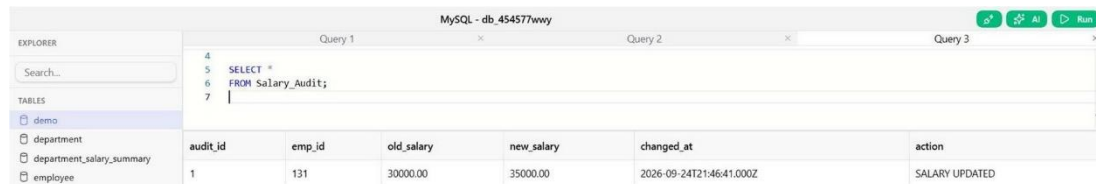
TEST AUDIT TRIGGER

```

UPDATE Employee SET salary = 35000
WHERE emp_id = 131;
SELECT *
FROM Salary_Audit;

```

OUTPUT:



The screenshot shows the MySQL Workbench interface. On the left, the 'EXPLORER' pane lists tables: demo, department, department_salary_summary, and employee. The main query editor shows a query: 'SELECT * FROM Salary_Audit;'. Below the editor, the results are displayed in a table format.

audit_id	emp_id	old_salary	new_salary	changed_at	action
1	131	30000.00	35000.00	2026-09-24T21:46:41.000Z	SALARY UPDATED

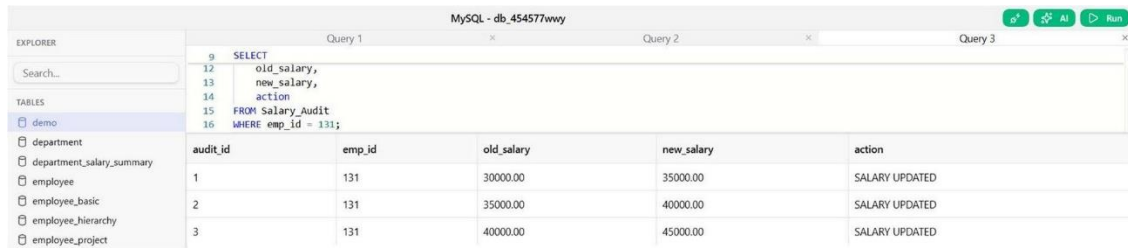
TEST MULTIPLE SALARY CHANGES

```

UPDATE Employee
SET salary = 40000
WHERE emp_id = 131;
UPDATE Employee
SET salary = 45000
WHERE emp_id = 131;
SELECT
    audit_id,
    emp_id,
    old_salary,
    new_salary,
    action
FROM Salary_Audit
WHERE emp_id = 131;

```

OUTPUT:



audit_id	emp_id	old_salary	new_salary	action
1	131	30000.00	35000.00	SALARY UPDATED
2	131	35000.00	40000.00	SALARY UPDATED
3	131	40000.00	45000.00	SALARY UPDATED

TEST EDGE CASE – SALARY UNCHANGED

```
UPDATE Employee
SET salary = 45000
WHERE emp_id = 131;
No new audit record is created
```

TEST EDGE CASE – NULL SALARY

```
DROP TRIGGER validate_employee_salary;
DELIMITER //
CREATE TRIGGER validate_employee_salary
BEFORE INSERT ON Employee
FOR EACH ROW
BEGIN
    IF NEW.salary IS NULL OR NEW.salary < 15000 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT =
        'Salary cannot be NULL and must be at least 15000';
    END IF;
END //
DELIMITER ;
INSERT INTO Employee
(emp_id, emp_name, gender, salary, hire_date, dept_id)
VALUES
(132, 'Null Salary', 'F', NULL, '2026-09-25', 1);
```

OUTPUT:

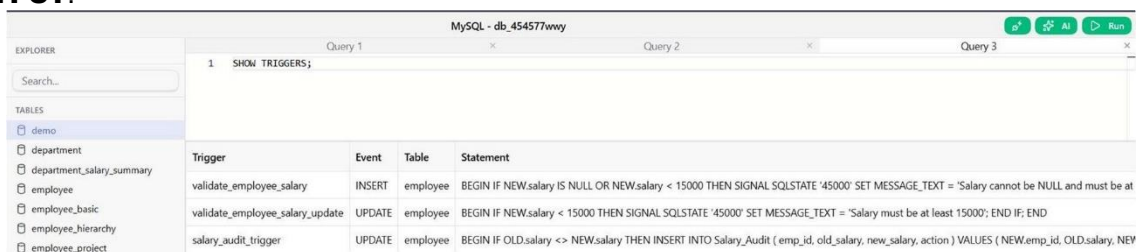


Trigger	Event	Table	Statement
validate_employee_salary	INSERT	employee	BEGIN IF NEW.salary IS NULL OR NEW.salary < 15000 THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Salary cannot be NULL and must be at least 15000';
validate_employee_salary_update	UPDATE	employee	BEGIN IF NEW.salary < 15000 THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Salary must be at least 15000'; END IF; END
salary_audit_trigger	UPDATE	employee	BEGIN IF OLD.salary <> NEW.salary THEN INSERT INTO Salary_Audit (emp_id, old_salary, new_salary, action) VALUES (NEW.emp_id, OLD.salary, NEW.salary, 'SALARY UPDATED');

CHECK ALL TRIGGERS

```
SHOW TRIGGERS;
```

OUTPUT:

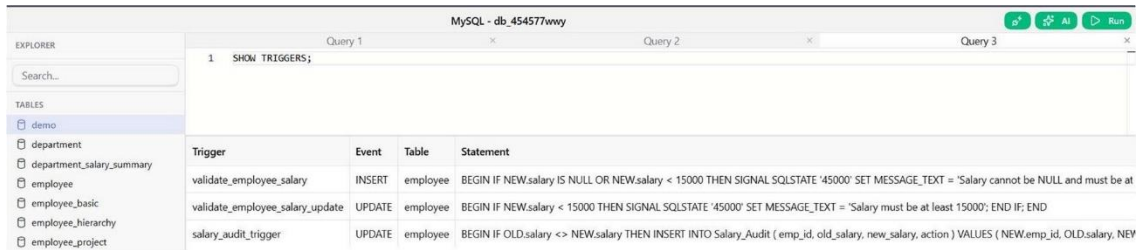


Trigger	Event	Table	Statement
validate_employee_salary	INSERT	employee	BEGIN IF NEW.salary IS NULL OR NEW.salary < 15000 THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Salary cannot be NULL and must be at least 15000';
validate_employee_salary_update	UPDATE	employee	BEGIN IF NEW.salary < 15000 THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Salary must be at least 15000'; END IF; END
salary_audit_trigger	UPDATE	employee	BEGIN IF OLD.salary <> NEW.salary THEN INSERT INTO Salary_Audit (emp_id, old_salary, new_salary, action) VALUES (NEW.emp_id, OLD.salary, NEW.salary, 'SALARY UPDATED');

CHECK ALL TRIGGERS

SHOW TRIGGERS;

OUTPUT:



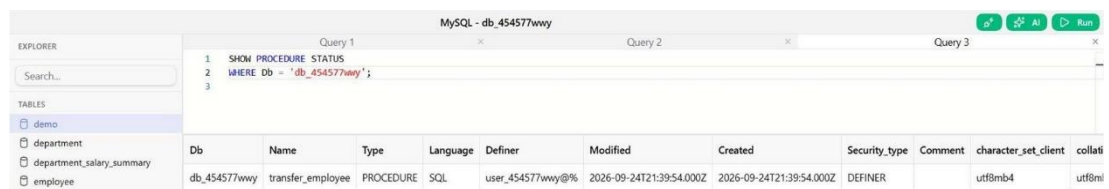
Trigger	Event	Table	Statement
validate_employee_salary	INSERT	employee	BEGIN IF NEW.salary IS NULL OR NEW.salary < 15000 THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Salary cannot be NULL and must be at least 15000'; END IF; END
validate_employee_salary_update	UPDATE	employee	BEGIN IF NEW.salary < 15000 THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Salary must be at least 15000'; END IF; END
salary_audit_trigger	UPDATE	employee	BEGIN IF OLD.salary <> NEW.salary THEN INSERT INTO Salary_Audit (emp_id, old_salary, new_salary, action) VALUES (NEW.emp_id, OLD.salary, NEW.salary, 'UPDATE'); END IF; END

CHECK STORED PROCEDURE

SHOW PROCEDURE STATUS

WHERE Db = ' db_454577wwy';

OUTPUT:



Db	Name	Type	Language	Definer	Modified	Created	Security_type	Comment	character_set_client	collation_client
db_454577wwy	transfer_employee	PROCEDURE	SQL	user_454577wwy@%	2026-09-24T21:39:54.000Z	2026-09-24T21:39:54.000Z	DEFINER		utf8mb4	utf8mb4_0900_ai_ci

